Übungsblatt 2: Speicherhierarchie und Caches

Mikrocomputertechnik 3 · 32 Orientierungspunkte · ca. 60–75 Minuten

Musterlösung

Aufgabe 1: Lokalität an einer Blockgrenze (4 Punkte)

Zwei unabhängige Lesefolgen, jeweils kalter Cache, 32-Byte-Lines, keine Verdrängung:

- A: `0x207C, 0x207C, 0x207C, 0x207C`
- B: `0x2078, 0x207C, 0x2080, 0x2084`

1. Welche Lokalität nutzt jede Folge? (2 P)
2. Geben Sie Hit/Miss je Zugriff und die vollständigen berührten Linebereiche an. Warum garantiert räumliche Lokalität nicht nur einen Miss? (2 P)

Musterlösung 1

A: zeitlich; Miss, Hit, Hit, Hit. B: räumlich; Miss, Hit, Miss, Hit. Linebereiche `0x2060` bis `0x207F` und `0x2080` bis `0x209F`. B überschreitet die Blockgrenze. Je 1 P für die beiden Lokalitäten, 1 P für Folgen und 1 P für Grenzen/Begründung.

Aufgabe 2: AMAT aus einer Messspur im Modell (5 Punkte)

Hitzeit zwei Takte, zusätzliche Misspenalty sechzig Takte. AMAT ist die mittlere Speicherzugriffszeit; alle Anfragen sind abgeschlossen.

1. 120 Anfragen ergeben 108 Hits und zwölf Misses. Bestimmen Sie Missrate und AMAT. Kontrollieren Sie durch Summieren der Einzelzeiten. (3 P)
2. Berechnen Sie AMAT bei 75 Prozent Hitrate. (1 P)
3. Ist das Verhältnis beider Werte bereits ein Programmspeedup? (1 P)

Musterlösung 2

1. $m = 12/120 = 0{,}1$, AMAT $= 2 + 0{,}1 \cdot 60 = 8$ Takte. Kontrolle: $(108 \cdot 2 + 12 \cdot 62)/120 = 960/120 = 8$. (3 P)
2. $2 + 0{,}25 \cdot 60 = 17$ Takte. (1 P)
3. $17/8 = 2{,}125$ vergleicht Zugriffszeiten; übrige Arbeit und Überlappung fehlen. (1 P)

Aufgabe 3: Andere Cachegeometrie (6 Punkte)

32-Bit-Byteadressen, 16 KiB Datenkapazität, 32 Byte je Line; Tags und Zustandsbits sind nicht in der Kapazität enthalten.

1. Bestimmen Sie Sets, Tag-, Index- und Offsetbits für Direct Mapped und 2-Way. (3 P)
2. Zerlegen Sie `0x3A5C91E6` für Direct Mapped und kontrollieren Sie durch Rückrechnung. (2 P)
3. Bestimmen Sie für die Dezimaladresse 1510 Blockbeginn, letztes Blockbyte und Offset. (1 P)

Musterlösung 3

Organisation	Sets	Tagbits	Indexbits	Offsetbits
Direct Mapped	512	18	9	5
2-Way	256	19	8	5

Je 1 P pro Zeile, 1 P: gleicher Offset wegen gleicher Linegröße. Adresse: Offset sechs, Index 143, Tag `0xE972`. Rückrechnung $59762 \cdot 16384 + 143 \cdot 32 + 6 = 979145190 =$ `0x3A5C91E6`. (2 P) Für 1510: Block 47, Beginn 1504, letztes Byte 1535, Offset sechs. (1 P)

Aufgabe 4: Konflikte und Ersatzreihenfolge (6 Punkte)

Cachegeometrie wie Aufgabe 3, beide Caches anfangs leer. Nur diese sieben Reads zählen: `0x3000, 0x7000, 0x3004, 0x7004, 0x3000, 0x7000, 0x3008`.

1. Erstellen Sie eine Tabelle mit DM-Index, DM-Tag und Hit/Miss für Direct Mapped sowie Hit/Miss für 2-Way. Bestimmen Sie beide Quoten und die Ursache zusätzlicher Misses. (4 P)
2. Separates leeres 3-Way-Set, Blockfolge J, K, L, J, M. Geben Sie die Ordnung nach dem zweiten J und das Opfer bei M an: FIFO nach Ladealter, LRU nach letzter Nutzung. (2 P)

Musterlösung 4

Schritt	DM-Index	DM-Tag	DM	2-Way
1	384	0	Miss	Miss
2	384	1	Miss	Miss
3	384	0	Miss	Hit
4	384	1	Miss	Hit
5	384	0	Miss	Hit
6	384	1	Miss	Hit
7	384	0	Miss	Hit

DM: null Hits, sieben Misses. 2-Way: fünf Hits, zwei Misses, $5/7 \approx 71{,}43\,\%$. Dort Set 128, Tags eins und drei. Die zusätzlichen DM-Misses sind Konflikte: Zwei Blöcke passen in die Gesamtkapazität. (4 P) FIFO: J, K, L; Opfer J. LRU: K, L, J; Opfer K. Nur LRU verändert beim Hit die Nutzungsordnung. (2 P)

Aufgabe 5: Writeback tritt vor dem letzten Zugriff auf (5 Punkte)

Direct Mapped wie Aufgabe 3, Write-Back und Write-Allocate, keine Puffer. Anfangs ist Block `0x3000` gültig und sauber; sein betrachtetes Wort ist zwei, auch in der unteren Ebene. Block `0x7000` fehlt im Cache und enthält unten acht.

Folge: Store sechs nach `0x3000`; Load `0x7000`; Load `0x3000`; Store elf nach `0x3000`.

1. Notieren Sie nach jedem Schritt Blockbasis, Wortwert, Dirty, unteren Wert bei `0x3000` und eine mögliche Writeback-Adresse. (3 P)
2. Aus welchen alten Informationen entsteht die Writeback-Adresse? (1 P)
3. Wie wirkt No-Write-Allocate bei einem Store-Miss? Folgt die Regel aus Write-Back? (1 P)

Musterlösung 5

Schritt	Blockbasis	Wert	Dirty	Unten bei 0x3000	Writeback
Store 6	`0x3000`	6	1	2	keiner
Load 0x7000	`0x7000`	8	0	6	`0x3000`
Load 0x3000	`0x3000`	6	0	6	keiner
Store 11	`0x3000`	11	1	6	keiner

Tabelle 3 P. Alte Basis $(0 \cdot 512 + 384) \cdot 32 =$ `0x3000` aus altem Tag und Index, nicht aus der neu angefragten Adresse. (1 P) No-Write-Allocate gibt den Store weiter, ohne eine neue Line zuzuteilen; diese Entscheidung ist von Write-Back getrennt. (1 P)

Aufgabe 6: Ein rechteckiges C-Array (6 Punkte)

`uint32_t feld[12][8]`, vier Byte je Element, ausgerichtete Basis `0x3000`. Zwei unabhängige Modellspuren laden genau einmal jedes Element: A zeilenweise, B spaltenweise. Keine anderen Zugriffe oder Optimierungen.

1. Geben Sie die Adressformel und die ersten vier Adressen beider Spuren an. (2 P)
2. Direct Mapped, 128 Byte Datenkapazität, 32-Byte-Lines, vor jeder Spur kalt: Bestimmen Sie Set-Index und vollständige Hit/Miss-Bilanzen. (3 P)
3. Wie ändert sich B bei 512 Byte Kapazität und sonst gleichen Bedingungen? (1 P)

Musterlösung 6

1. $a_{r,c} = \texttt{0x3000} + 4(8r + c)$. A: `3000, 3004, 3008, 300C`; B: `3000, 3020, 3040, 3060`, jeweils hexadezimal. (2 P)
2. Vier Sets, eine Arrayzeile je Line; Index $(384 + r) \bmod 4 = r \bmod 4$. A: zwölf Misses, 84 Hits. B: 96 Misses, null Hits; je Spalte verdrängen zwölf Zeilen einander in vier Sets. (3 P)
3. Sechzehn Sets halten alle zwölf Zeilen: B hat zwölf Misses und 84 Hits. (1 P)

Optional: am Rechner prüfen

Erzeugen Sie die beiden Adressspuren aus Aufgabe 6 in einem kurzen Programm. Vergleichen Sie sie mit einem verfügbaren Cache-Simulator bei 128 und 512 Byte. Zählen Sie ausschließlich diese Loads; dokumentieren Sie Anfangszustand und Einstellungen.